Loan Default Risk Analysis

Notebook code organised by analysis question | Dataset: Loan_default.csv (255,347 rows, 18 columns)

Each question is followed by the code from the notebook and its output. Cells marked **(added)** were not in the original notebook and have no output shown - run them to see results. Cells that make charts show a short description of the chart instead of the image.

1. What libraries are required for this loan default risk analysis?

Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

2. How can we import the loan dataset into Python?

Code

```
df = pd.read_csv(r"D:\.41-f04c858169c2cdf929d0fc0771c3603b\Loan_default.csv")
```

3. What does the loan dataset contain?

Code

```
df.head()
```

Output

```
    LoanID      Age  Income  LoanAmount  CreditScore  MonthsEmployed  NumCredit...
0  I38PQUQS96   56   85994    50587        520            80
1  HPSK72WA7R   69   50432   124440        458            15
2  C1OZ6DPJ8Y   46   84208   129188        451            26
3  V2KKSFM3UN   32   31713    44799        743             0
4  EY08JDHTZP   60   20437     9139        633             8
```

Code

```
df.columns.tolist()
```

Output

```
['LoanID', 'Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEmployed',
 'NumCreditLines', 'InterestRate', 'LoanTerm', 'DTIRatio', 'Education',
 'EmploymentType', 'MaritalStatus', 'HasMortgage', 'HasDependents',
 'LoanPurpose', 'HasCoSigner', 'Default']
```

4. How many rows and columns are in the dataset?

Code (added)

```
df.shape
```

From df.info() below: 255,347 rows and 18 columns, so df.shape returns (255347, 18).

5. What are the data types and basic characteristics of each column?

Code

```
df.info()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 255347 entries, 0 to 255346
Data columns (total 18 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
```

```
 0   LoanID          255347 non-null  str
 1   Age             255347 non-null  int64
 2   Income          255347 non-null  int64
 3   LoanAmount      255347 non-null  int64
 4   CreditScore     255347 non-null  int64
 5   MonthsEmployed  255347 non-null  int64
 6   NumCreditLines  255347 non-null  int64
 7   InterestRate    255347 non-null  float64
 8   LoanTerm        255347 non-null  int64
 9   DTIRatio        255347 non-null  float64
10   Education       255347 non-null  str
11   EmploymentType  255347 non-null  str
12   MaritalStatus   255347 non-null  str
13   HasMortgage     255347 non-null  str
14   HasDependents   255347 non-null  str
15   LoanPurpose     255347 non-null  str
16   HasCoSigner     255347 non-null  str
17   Default         255347 non-null  int64
dtypes: float64(2), int64(8), str(8)
memory usage: 35.1 MB
```

Code

```
df.describe()
```

Output

```
             Age         Income      LoanAmount   CreditScore  MonthsEmployed
count  255347.000   255347.000   255347.000   255347.000   255347.000
mean       43.498    82499.305   127578.866      574.264       59.542
std        14.990    38963.014    70840.706      158.904       34.643
min        18.000    15000.000     5000.000      300.000        0.000
25%        31.000    48825.500    66156.000      437.000       30.000
50%        43.000    82466.000   127556.000      574.000       60.000
75%        56.000   116219.000   188985.000      712.000       90.000
max        69.000   149999.000   249999.000      849.000      119.000
```

6. Are there any missing values in the dataset?

Code

```
df.isnull().sum()
```

Output

```
LoanID            0
Age               0
Income            0
LoanAmount        0
CreditScore       0
MonthsEmployed    0
NumCreditLines    0
InterestRate      0
LoanTerm          0
DTIRatio          0
Education         0
EmploymentType    0
MaritalStatus     0
HasMortgage       0
HasDependents     0
LoanPurpose       0
HasCoSigner       0
Default           0
dtype: int64
```

No missing values in any column.

7. Are there any duplicate loan applications in the dataset?

Code (added)

```
# fully duplicated rows
print("Duplicate rows:", df.duplicated().sum())

# repeated loan IDs
print("Duplicate LoanIDs:", df["LoanID"].duplicated().sum())
```

Added code - this check was not in the original notebook, so run it to see the result.

8. How can we rename the columns to make them easier to work with?

Code

```
df = df.rename(columns={
    'LoanID': 'loan_id',
    'LoanAmount': 'loan_amount',
    'CreditScore': 'credit_score',
    'MonthsEmployed': 'months_employed',
    'NumCreditLines': 'num_credit_lines',
    'InterestRate': 'interest_rate',
    'LoanTerm': 'loan_term',
    'DTIRatio': 'dti_ratio',
    'EmploymentType': 'employment_type',
    'MaritalStatus': 'marital_status',
    'HasMortgage': 'has_mortgage',
    'HasDependents': 'has_dependents',
    'LoanPurpose': 'loan_purpose',
    'HasCoSigner': 'has_co_signer'
})
```

Code

```
df.head()
```

Output

	loan_id	Age	Income	loan_amount	credit_score	months_employed	num_cred...
0	I38PQUQS96	56	85994	50587	520	80	
1	HPSK72WA7R	69	50432	124440	458	15	
2	C1OZ6DPJ8Y	46	84208	129188	451	26	
3	V2KKSFM3UN	32	31713	44799	743	0	
4	EY08JDHTZP	60	20437	9139	633	8	

9. What is the overall historical default rate?

Code

```
df["Default"].value_counts(normalize=True) * 100
```

Output

```
Default
0    88.387175
1    11.612825
Name: proportion, dtype: float64
```

Code

```
Default_rate = df["Default"].mean() * 100
print(f"overall default rate: {Default_rate: .2f}%")
```

Output

```
overall default rate:  11.61%
```

10. How does historical default rate differ between defaulted and non-defaulted applications?

Code

```
df["Default"].value_counts()
```

Output

```
Default
0    225694
1     29653
Name: count, dtype: int64
```

Code

```
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Default")
plt.title("Default_loan_Distribution")
plt.show()
```

Chart: a bar plot with ~225,694 non-defaults vs ~29,653 defaults - the classes are heavily imbalanced (about 88% vs 12%).

11. How does age differ between customers who defaulted and those who did not?

Code

```
df.groupby("Default")["Age"].mean()
```

Output

```
Default
0    44.409962
1    36.559539
Name: Age, dtype: float64
```

Defaulters are about 8 years younger on average.

12. Which age groups have the highest historical default rates?

Code

```
df["AgeGroup"] = pd.cut(df["Age"], bins=[0, 25, 35, 45, 55, 100],
                        labels=["18-25", "26-35", "36-45", "46-55", "+56"])

age_default = (df.groupby("AgeGroup", observed=True)["Default"]
                 .mean() * 100).sort_values(ascending=False)
print(age_default)
```

Output

```
AgeGroup
18-25    20.760714
26-35    16.078368
36-45    11.548151
46-55     8.449988
+56       5.502152
Name: Default, dtype: float64
```

13. How does income differ between defaulted and non-defaulted applications?

Code

```
df["Income"].describe()
```

Output

```
count    255347.000000
mean      82499.304597
std       38963.013729
min       15000.000000
25%       48825.500000
50%       82466.000000
```

```
75%      116219.000000
max      149999.000000
Name: Income, dtype: float64
```

Code (added)

```
df.groupby("Default")["Income"].mean()
```

Added code - the original notebook only described income overall; this splits it by default status.

14. Which income groups have the highest historical default rates?

Code

```
df['IncomeGroup'] = pd.qcut(
    df['Income'],
    q=4,
    labels=['Low Income', 'Lower-Middle Income', 'Upper-Middle Income', 'High Income']
)

income_default = (
    df.groupby('IncomeGroup', observed=True)['Default']
      .mean() * 100
).sort_values(ascending=False)

print(income_default)
```

Output

```
IncomeGroup
Low Income             17.380203
Lower-Middle Income    10.537776
Upper-Middle Income     9.512994
High Income             9.020271
Name: Default, dtype: float64
```

15. How does loan amount differ between defaulted and non-defaulted applications?

Code

```
df.groupby("Default")["loan_amount"].mean()
```

Output

```
Default
0    125353.656017
1    144515.311469
Name: loan_amount, dtype: float64
```

16. How does the loan-to-income ratio differ between defaulted and non-defaulted applications?

Code

```
df["loan_income"] = df["loan_amount"] / df["Income"]
```

Code

```
df.groupby("Default")["loan_income"].mean()
```

Output

```
Default
0    2.036172
1    3.253000
Name: loan_income, dtype: float64
```

Code

```
df["loan_income"].describe()
```

Output

```
count    255347.000000
```

```
mean          2.177480
std           2.178357
min           0.033749
25%           0.803628
50%           1.546720
75%           2.671744
max          16.532460
Name: loan_income, dtype: float64
```

17. Which loan-to-income groups have the highest historical default rates?

Code

```
df['LoanIncomeGroup'] = pd.qcut(
    df['loan_income'],
    q=4,
    labels=['Low', 'Moderate', 'High', 'Very High']
)

loan_income_default = (
    df.groupby('LoanIncomeGroup', observed=True)['Default']
      .mean() * 100
).sort_values(ascending=False)

print(loan_income_default)
```

Output

```
LoanIncomeGroup
Very High    18.691355
High         10.628799
Moderate      9.174930
Low           7.956201
Name: Default, dtype: float64
```

18. How does credit score differ between defaulted and non-defaulted applications?

Code

```
df.groupby("Default")["credit_score"].mean()
```

Output

```
Default
0    576.232270
1    559.286143
Name: credit_score, dtype: float64
```

Code

```
plt.figure(figsize=(4, 6))
sns.boxplot(data=df, x="Default", y="credit_score")
plt.xlabel("Default (0=no, 1=yes)")
plt.title("credit score distribution by default status")
plt.ylabel("credit score")
plt.show()
```

Chart: box plot - the defaulted group sits slightly lower (median roughly 550 vs 575), but the two distributions overlap heavily.

19. Which credit score groups have the highest historical default rates?

Code

```
df['CreditScoreGroup'] = pd.qcut(
    df['credit_score'],
    q=4,
    labels=['Low', 'Moderate', 'High', 'Very High']
)

credit_default = (
    df.groupby('CreditScoreGroup', observed=True)['Default']
      .mean() * 100
).sort_values(ascending=False)

print(credit_default)
```

Output

```
CreditScoreGroup
Low          13.055815
Moderate     11.894425
High         11.290222
Very High    10.197141
Name: Default, dtype: float64
```

20. Which employment types have the highest historical default rates?

Code

```
df.groupby("employment_type")["Default"].mean().mul(100).sort_values(ascending=False)
```

Output

```
employment_type
Unemployed       13.552895
Part-time        11.965213
Self-employed    11.462029
Full-time         9.463366
Name: Default, dtype: float64
```

21. Which loan purposes have the highest historical default rates?

Code

```
df.groupby("loan_purpose")["Default"].mean().mul(100).sort_values(ascending=False)
```

Output

```
loan_purpose
Business     12.326017
Auto         11.881441
Education    11.838055
Other        11.788506
Home         10.234762
Name: Default, dtype: float64
```

22. How does the debt-to-income ratio differ between defaulted and non-defaulted applications?

Code

```
df.groupby("Default")["dti_ratio"].mean()
```

Output

```
Default
0    0.498602
1    0.512467
Name: dti_ratio, dtype: float64
```

A very small gap - DTI alone does not separate defaulters well in this dataset.

23. How can we create age groups to compare historical default rates?

Code

```
df["AgeGroup"] = pd.cut(df["Age"], bins=[0, 25, 35, 45, 55, 100],
                        labels=["18-25", "26-35", "36-45", "46-55", "+56"])
```

Same cell as used for the age-group results above. Bins: 18-25, 26-35, 36-45, 46-55, 56+.

24. How can we create income groups to compare historical default rates?

Code

```
df['IncomeGroup'] = pd.qcut(
    df['Income'],
    q=4,
    labels=['Low Income', 'Lower-Middle Income', 'Upper-Middle Income', 'High Income']
)
```

qcut splits income into four equal-sized quartiles (each holds ~25% of applications).

25. How can we create loan-to-income groups to measure financial burden?

Code

```
df["loan_income"] = df["loan_amount"] / df["Income"]

df['LoanIncomeGroup'] = pd.qcut(
    df['loan_income'],
    q=4,
    labels=['Low', 'Moderate', 'High', 'Very High']
)
```

Loan-to-income = loan amount divided by annual income; it measures how heavy the loan is relative to earnings.

26. How can we create credit score groups to compare historical default rates?

Code

```
df['CreditScoreGroup'] = pd.qcut(
    df['credit_score'],
    q=4,
    labels=['Low', 'Moderate', 'High', 'Very High']
)
```

27. How can we create a transparent rule-based review score?

Code

```
df['ReviewScore'] = (
    (df['AgeGroup'] == '18-25').astype(int)
    + (df['IncomeGroup'] == 'Low Income').astype(int)
    + (df['LoanIncomeGroup'] == 'Very High').astype(int)
    + (df['CreditScoreGroup'] == 'Low').astype(int)
    + (df['employment_type'] == 'Unemployed').astype(int)
)

df['ReviewScore'].value_counts().sort_index()
```

Output

```
ReviewScore
0    81411
1    88586
2    56038
3    23648
4     5272
5      392
Name: count, dtype: int64
```

Each application gets 1 point per risk flag (age 18-25, low income, very high loan-to-income, low credit score, unemployed), so the score runs from 0 to 5.

28. How does historical default rate change as the review score increases?

Code

```
df.groupby("ReviewScore")["Default"].mean().mul(100).round(2)
```

Output

```
ReviewScore
0     7.00
1     9.83
2    14.70
3    22.14
4    30.50
5    42.60
Name: Default, dtype: float64
```

The default rate rises steadily with every extra flag - from 7.00% at score 0 to 42.60% at score 5.

29. How can we classify applications into review categories?

Code

```
df['ReviewCategory'] = pd.cut(
    df['ReviewScore'],
    bins=[-1, 1, 3, 5],
    labels=['Standard', 'Manual Review', 'High Priority Review']
)

df['ReviewCategory'].value_counts().sort_index()
```

Output

```
ReviewCategory
Standard               169997
Manual Review           79686
High Priority Review     5664
Name: count, dtype: int64
```

Score 0-1 = Standard, 2-3 = Manual Review, 4-5 = High Priority Review.

30. Which loan applications should be flagged for manual review?

Code

```
df["Manual_review"] = df["ReviewScore"] >= 2
df["Manual_review"].value_counts()
```

Output

```
Manual_review
False    169997
True      85350
Name: count, dtype: int64
```

Code

```
manual_review = df[df["Manual_review"] == True].copy()
manual_review.shape
```

Output

```
(85350, 26)
```

Code

```
manual_review = df[
    [
        'loan_id', 'Age', 'Income', 'loan_amount', 'credit_score',
        'dti_ratio', 'employment_type', 'loan_purpose',
```

```
        'ReviewScore', 'ReviewCategory', 'Manual_review'
    ]
].copy()

manual_review.head()
```

Output

	loan_id	Age	Income	loan_amount	credit_score	dti_ratio	employment_type
0	I38PQUQS96	56	85994	50587	520	0.44	Full-time
1	HPSK72WA7R	69	50432	124440	458	0.68	Full-time
2	C1OZ6DPJ8Y	46	84208	129188	451	0.31	Unemployed
3	V2KKSFM3UN	32	31713	44799	743	0.23	Full-time
4	EY08JDHTZP	60	20437	9139	633	0.73	Unemployed

Applications with ReviewScore >= 2 (85,350 of them) are flagged for manual review. Note: the column-selection cell above runs on the full df, so the head() rows shown are not filtered - re-run the filter cell below to get only flagged loans.

Code

```
manual_review = df[df['Manual_review'] == True].copy()
manual_review.shape
```

Output

```
(85350, 26)
```

31. How many applications fall into each review category?

Code

```
manual_review['ReviewCategory'].value_counts()
```

Output

```
ReviewCategory
Manual Review           79686
High Priority Review     5664
Standard                    0
Name: count, dtype: int64
```

Code

```
manual_review['ReviewCategory'].unique()
```

Output

```
['Manual Review', 'High Priority Review']
Categories (3, str): ['Standard' < 'Manual Review' < 'High Priority Review']
```

Across all 255,347 applications: Standard 169,997 | Manual Review 79,686 | High Priority Review 5,664.

32. What are the key business insights from the loan default analysis?

- **Baseline:** 11.61% of loans defaulted (29,653 of 255,347). The data is imbalanced - roughly 88% non-default vs 12% default.
- **Age is the strongest single driver:** default falls from 20.76% (18-25) to 5.50% (56+). Defaulters average 36.6 years vs 44.4 for non-defaulters.

33. How can we prepare the analysis-ready dataset for Power BI?

Code (added)

```
powerbi_df = df.copy()

# Convert category-dtype columns to plain text so Power BI reads them cleanly
for col in ["AgeGroup", "IncomeGroup", "LoanIncomeGroup",
            "CreditScoreGroup", "ReviewCategory"]:
    powerbi_df[col] = powerbi_df[col].astype(str)

# Boolean flag -> 1/0
powerbi_df["Manual_review"] = powerbi_df["Manual_review"].astype(int)

# Consistent, readable column names for the dashboard
powerbi_df = powerbi_df.rename(columns={
    "Age": "age", "Income": "income", "Default": "default",
    "AgeGroup": "age_group", "IncomeGroup": "income_group",
    "LoanIncomeGroup": "loan_income_group",
    "CreditScoreGroup": "credit_score_group",
    "ReviewScore": "review_score", "ReviewCategory": "review_category",
    "Manual_review": "manual_review", "loan_income": "loan_to_income_ratio",
})

# Final checks
print(powerbi_df.shape)
print("Missing values:", powerbi_df.isnull().sum().sum())
powerbi_df.head()
```

Added code. Run it and confirm there are no missing values before exporting.

34. How can we export the final dataset for Power BI?

Code (added)

```
powerbi_df.to_csv("Loan_Default_PowerBI.csv", index=False)
print("Exported:", powerbi_df.shape[0], "rows")
```

In Power BI Desktop: Home > Get data > Text/CSV > select Loan_Default_PowerBI.csv > Load. Useful visuals: default rate by age_group, income_group and employment_type; a count of loans by review_category; default rate by review_score.

Appendix - fix for two cells in the original notebook

In the original notebook, the loan-term and interest-rate cells called .value_counts() on the group means, which counts how often each rate value appears instead of ranking the groups, so the output was not useful (that is why it showed '1' next to each value). Corrected versions:

Code (added)

```
# default rate by loan term, highest first
df.groupby("loan_term")["Default"].mean().mul(100).sort_values(ascending=False)

# interest rate has ~785 unique values, so bin it first
df["InterestRateGroup"] = pd.qcut(df["interest_rate"], q=4,
                                  labels=["Low", "Moderate", "High", "Very High"])
(df.groupby("InterestRateGroup", observed=True)["Default"]
   .mean().mul(100).sort_values(ascending=False))
```